

Rock Paper Scissors

A web game built with Java OOP, Spring Boot and Vue 3

Java 17+ | Spring Boot 3 | Vue 3 | Maven | JUnit 5

What the project does

Set up a game
Enter your name and pick Easy or Hard

Play a round
Click Rock, Paper or Scissors

Track the score
Wins, losses and draws update live

Review history
Every round listed, newest first

Tech stack

Java 17+ Game logic, OOP model

Spring Boot 3 REST API + web server

Vue 3 Reactive browser UI

Maven Build and dependencies

JUnit 5 Unit tests for rules & AI

Layered architecture



Vue 3 UI

index.html sends clicks as JSON



Controller

REST endpoints, no game logic



Service

Creates, stores and runs games



Game model

Rules, score and round history



Players & AI

Human, Computer, strategies

REST API

```
POST /api/games          new game
POST /api/games/{id}/play    play a round
POST /api/games/{id}/reset    reset score
GET  /api/games/{id}     current state
```

Why layers?

Each layer has one job. The game core is plain Java with no Spring code, so it could power a console or desktop app too.

Object-oriented design



Abstraction

player › Player
strategy › MoveStrategy

Abstract class / interface hide how a move is chosen



Encapsulation

model › Scoreboard, Game

Private fields, changed only via record() / playRound()



Inheritance

player › HumanPlayer, ComputerPlayer

Both extend Player and override chooseMove()



Polymorphism

Player.chooseMove()
MoveStrategy.nextMove()

Same call, different behaviour per subclass



Composition

model › Game
player › ComputerPlayer

Game has players + score; ComputerPlayer has a strategy



Strategy pattern

strategy › RandomStrategy, FrequencyStrategy

Easy / Hard AI swapped without changing ComputerPlayer

The computer AI and running it



Easy RandomStrategy
Picks Rock, Paper or Scissors at random



Hard FrequencyStrategy
Counts your moves and plays what beats your favourite

New AI level = one new MoveStrategy class + one Difficulty constant

Run it

1. **Install Java 17+ and Maven**
2. **Start the server**
   `mvn spring-boot:run`
3. **Open the game**
   `http://localhost:8080`

Tests: mvn test · 3 JUnit tests pass